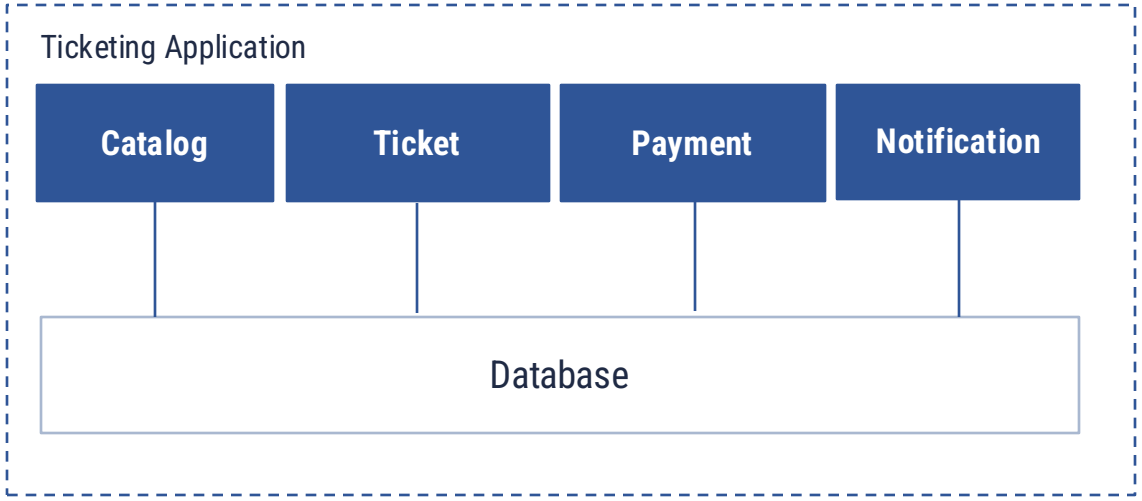
Introduction to Microservices

IF3110 – Web-based Application Development
Week 11 · Microservice architecture

School of Electrical Engineering and Informatics
Institut Teknologi Bandung

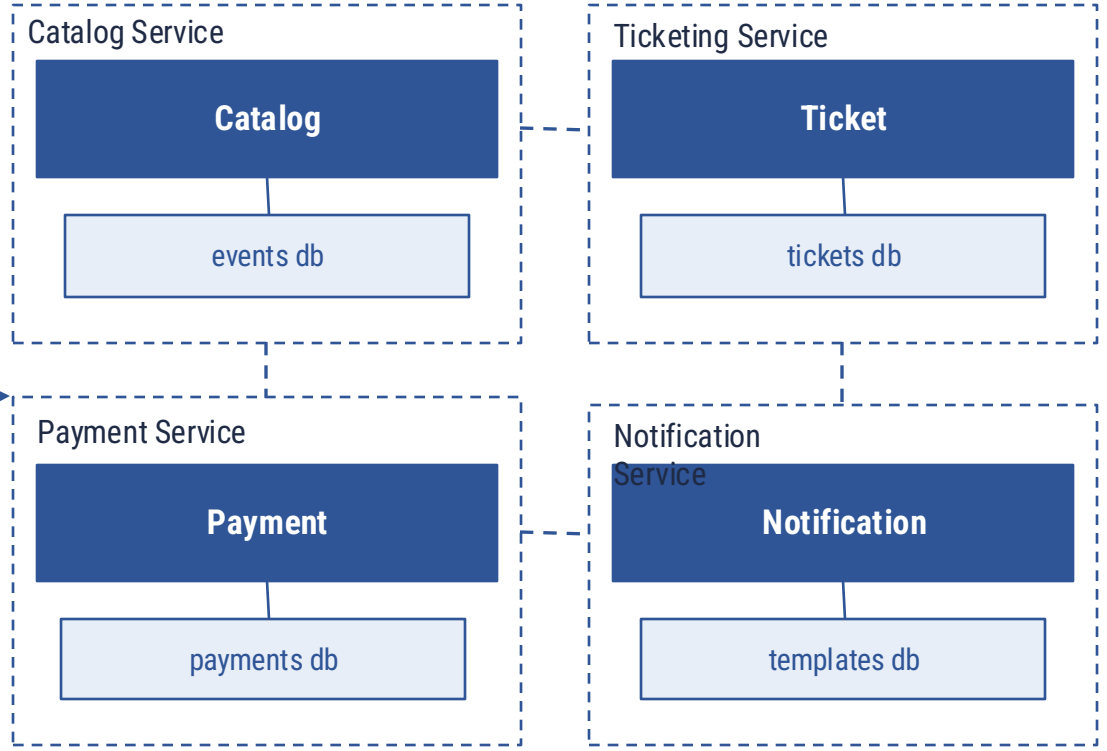
Campus Ticketing Application

WEEK 10 · SINGLE APPLICATION



One codebase, one build, one deploy.

TODAY · ONE APPLICATION PER BUSINESS CAPABILITY



Dashed lines: calls and messages over the network

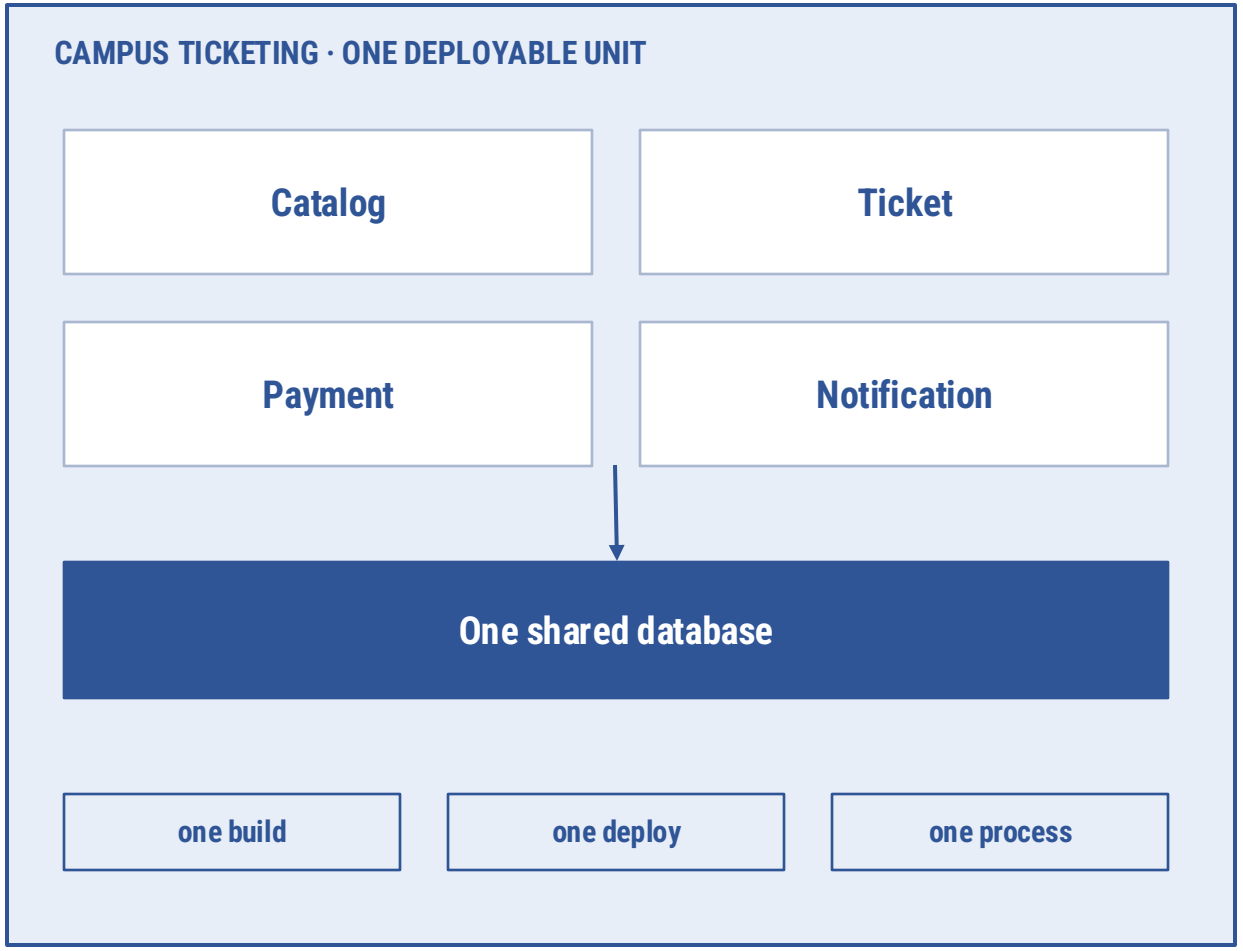
All Modules in one deployment as Week 10. Now each part is its own deployable application.

01

The monolith and its limits

Start from what most systems are, and why that is often fine.

What is a monolith?



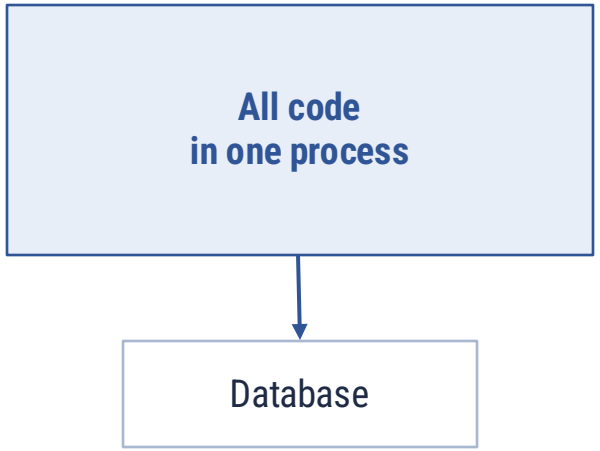
A system in which all functionality must be deployed together.

- Microservices are usually discussed as the alternative to the monolith.
- "Monolith" refers to a single, unified unit of deployment.
- It says nothing about code quality: a monolith can be clean and well structured.

The defining property is how a system is deployed, not how big it is.

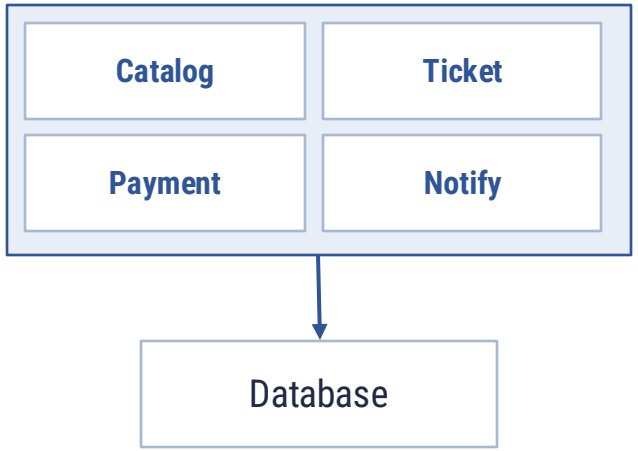
Three kinds of monolith

Single-process monolith



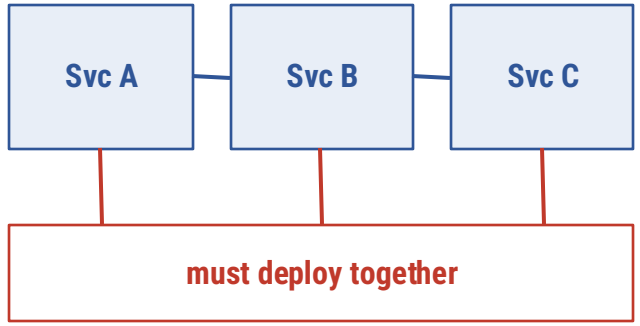
Simplest to build, run and debug. The common starting point.

Modular monolith



Separate modules, one deploy. A strong, often overlooked default.

Distributed monolith



Many services, but every change ships together. Costs of both, benefits of neither.

Splitting into services does not guarantee independence.

Monoliths are not bad

01

Simple deployment

One artifact to build, ship and roll back.

02

Easier debugging

One process, one log, one stack trace.

03

Simpler end-to-end testing

Start one app and test the whole flow.

04

Direct code reuse

Need shared logic? Just call the function.

05

No network between modules

In-process calls are fast and do not fail halfway.

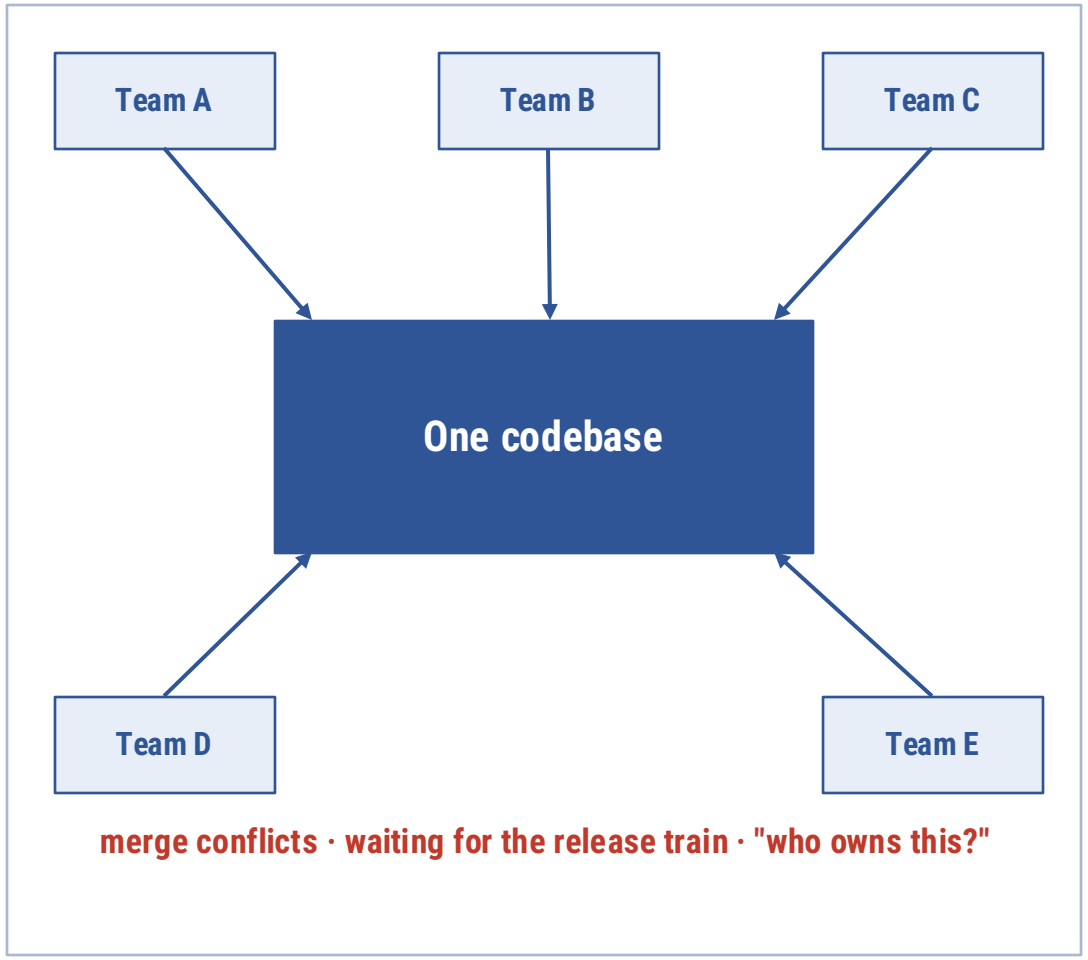
06

Fast start for small teams

Ideal for early products and changing requirements.

A monolith is a sensible default, not a legacy mistake.

Monoliths and delivery contention



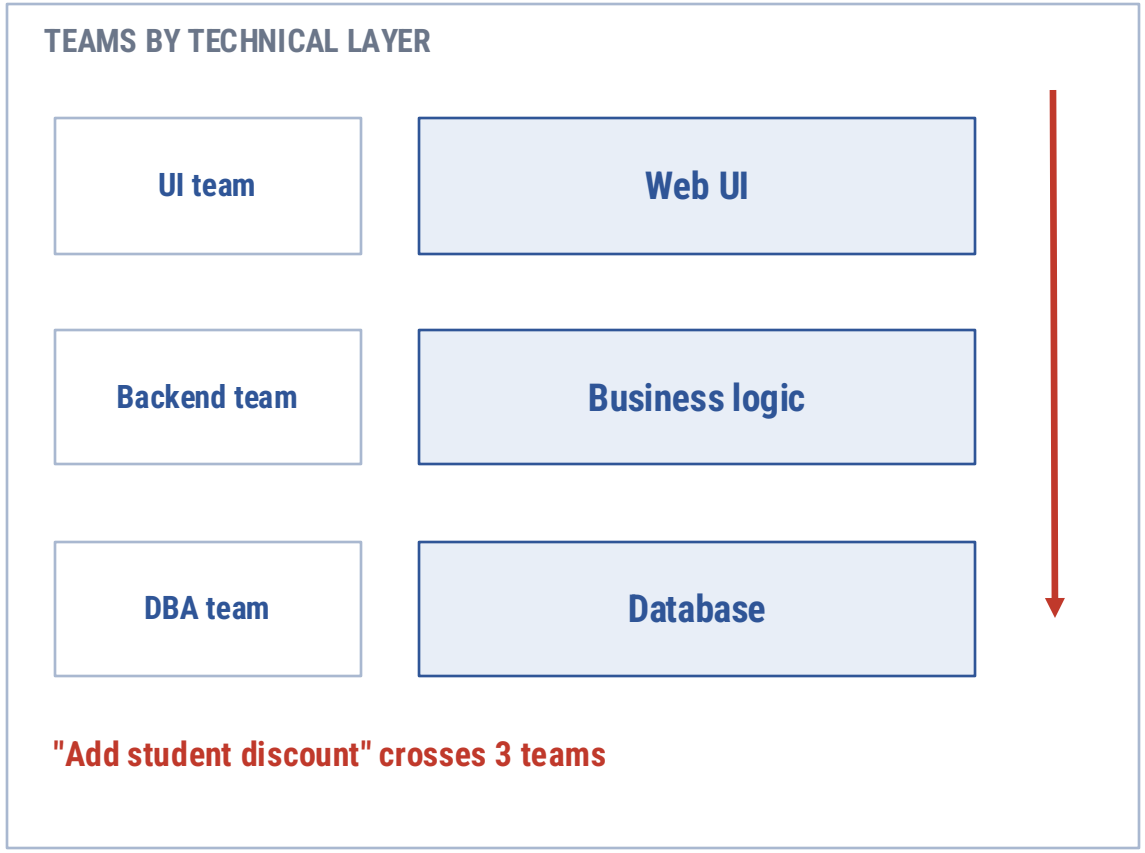
	Contention risk	Ownership	Deploy flexibility
Monolith	Higher: shared codebase	Often ambiguous	Limited: everything ships together
Microservices	Lower: isolated services	Clear boundaries	Greater: independent deploys

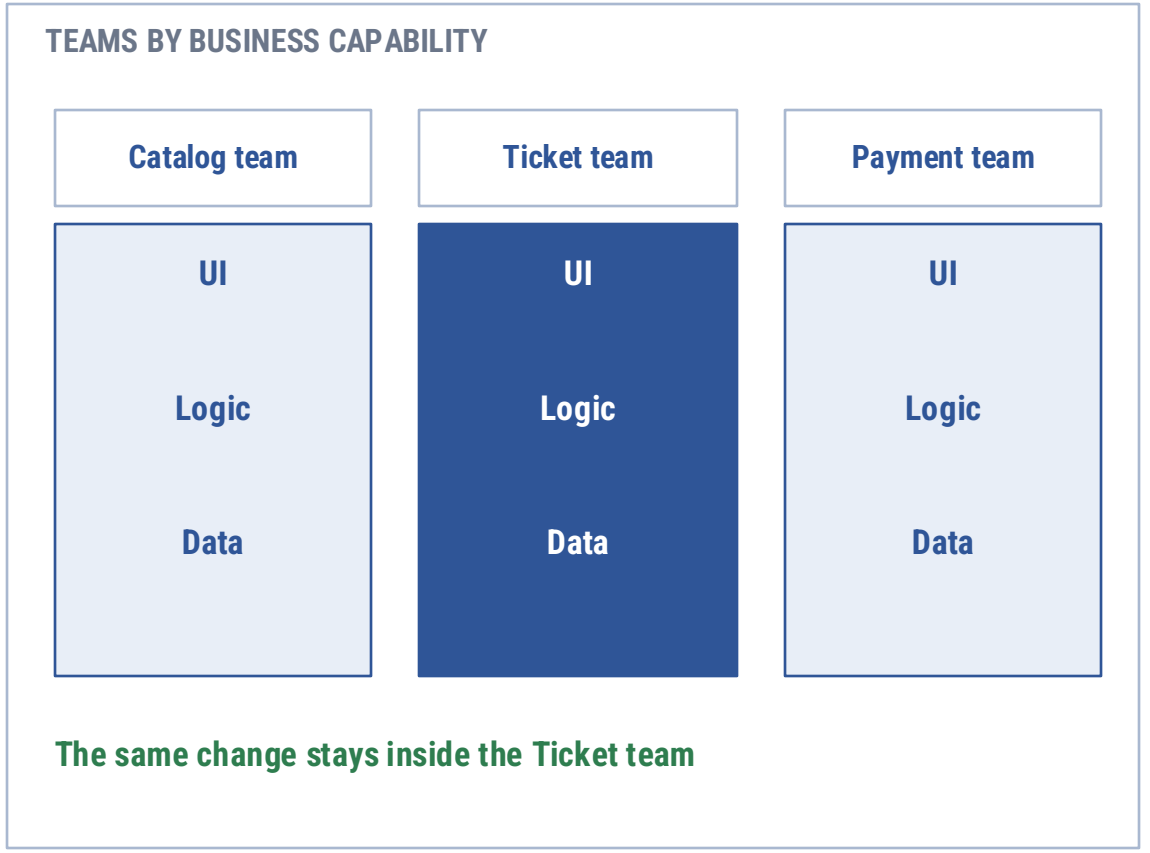
Neither column is guaranteed. Microservices give you concrete boundaries to draw ownership lines around.

Delivery contention is a people problem that architecture can make better or worse.

Conway's Law

Organizations design systems that mirror their own communication structure. — Melvin Conway, 1968 (paraphrased)





Inverse Conway maneuver: shape the teams the way you want the architecture to look.

02

Why microservices?

Every benefit is an answer to a specific pain. Start from the pain.

A growth story: campus ticketing

Year 1

Ticketing app

- 3 developers, 1 repository
- Deploy whenever a feature is ready
- Hundreds of users per event

Year 3

Same app, 10× the code, 30 developers in 5 teams

- **Release train:** one weekly deploy; a bug in Notification blocks Payment's fix.
- **Flash sale:** ticket checkout overloads the app and event browsing goes down too.
- **Scaling:** the only option is to copy the whole app, including idle parts.
- **Ownership:** nobody is sure who can change the shared tables.

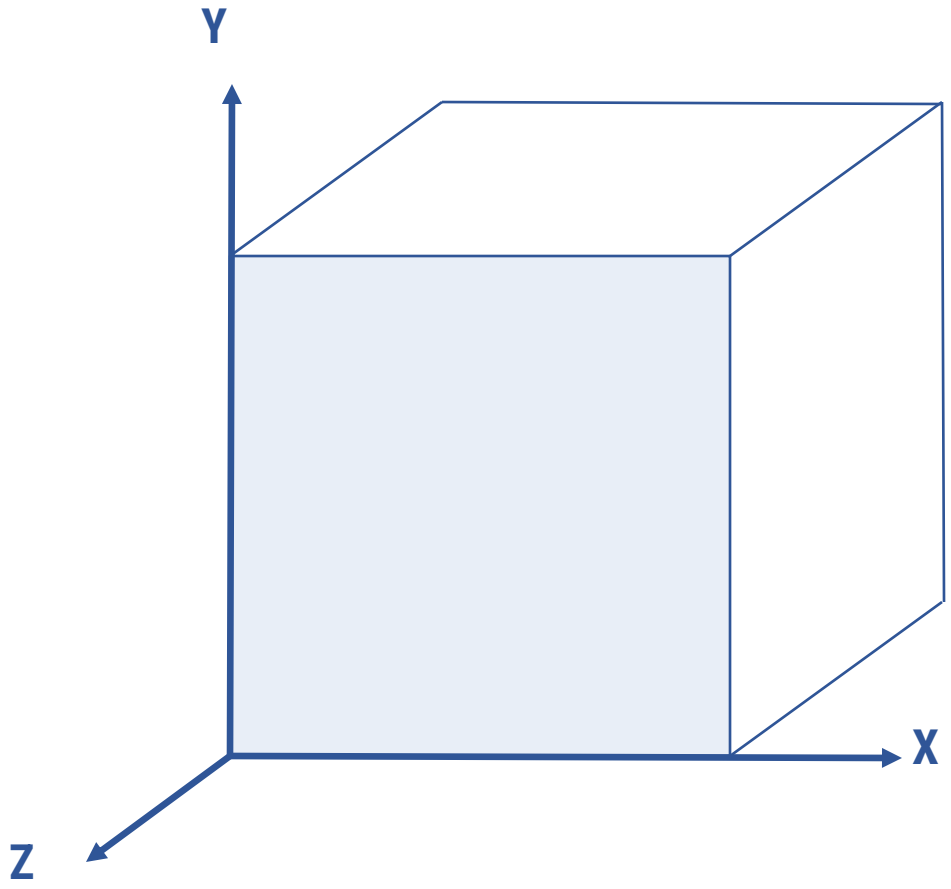
Nothing is wrong with the code. The system outgrew one deployable unit.

Five reasons teams move to microservices

YEAR-3 PAIN	WHAT MICROSERVICES OFFER	
Teams block each other	Team autonomy	Each team owns a few services end to end.
Release train, risky deploys	Independent deployment	Small changes ship alone; rollback is cheap.
Flash sale needs a whole-app copy	Targeted scaling	Scale only the Ticket service.
One bug takes the site down	Fault isolation	A failing service degrades one feature, if designed for it.
Stuck on one stack	Technology choice	Pick the right tool per service, e.g., a search engine for Catalog.

No pain, no reason: each benefit answers a specific problem.

The scale cube



X-axis · clone

Run N identical copies behind a load balancer.

Ticketing: 4 copies of the whole app.

Y-axis · split by function

Different services do different jobs. This is microservices.

Ticketing: Catalog, Ticket, Payment as separate services.

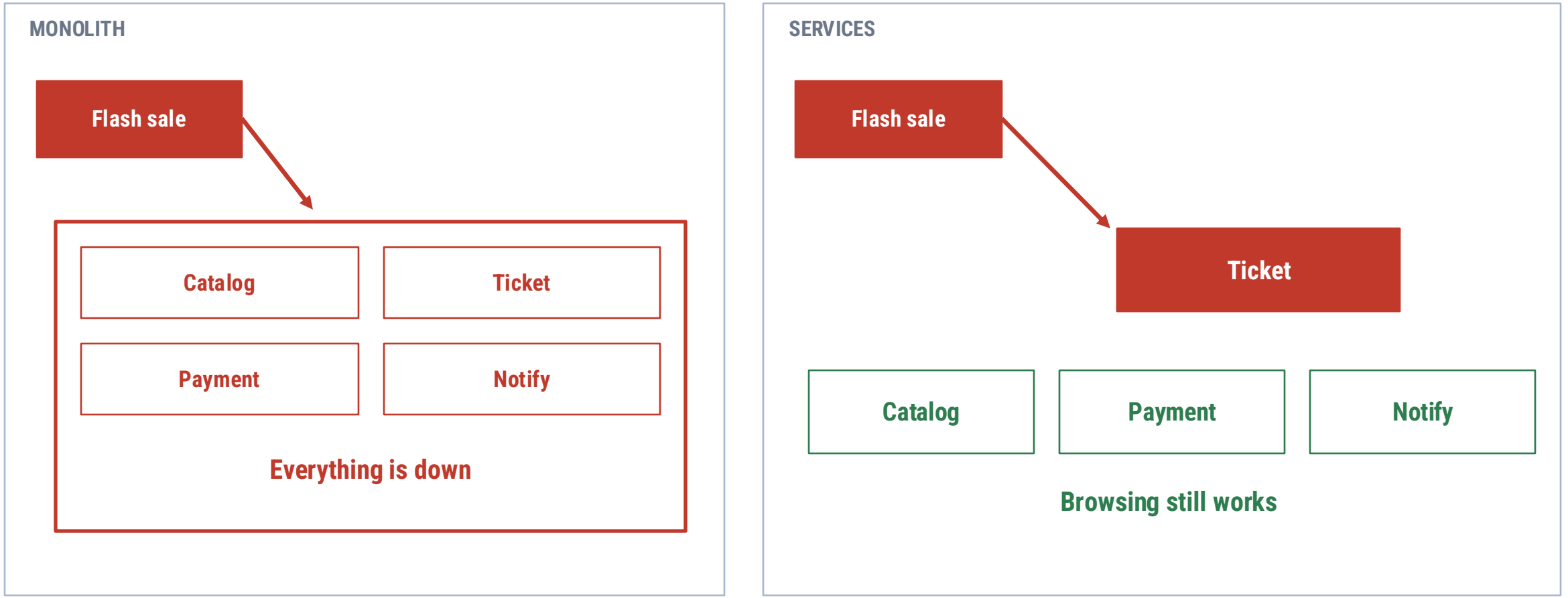
Z-axis · partition data

Each copy serves a subset of the data.

Ticketing: shard tickets by event or by campus.

Microservices are Y-axis scaling. They combine with X and Z, not replace them.

Fault isolation



Only if Catalog does not wait on Ticket synchronously. See section 7.

Isolation is a design property, not a free gift from splitting.

Why the big companies did it

Amazon

Monolith → services, 2001–2006

- Started as one application ("Obidos").
- By 2001 the front end could not scale with the business.
- Rule: a service owns its data; others use its interface only.
- Teams own a service end to end: "you build it, you run it".

Netflix

Data center monolith → cloud services, 2008–2016

- Aug 2008: database corruption stopped DVD shipping for 3 days.
- Rebuilt as many small, horizontally scalable services on AWS.
- Did not simply lift and shift the monolith.
- Finished the migration in January 2016.

Both moved because of scale: many teams and heavy load, not fashion.

When the "why" did not hold

Segment · 2018

100+ destination services → one service

- Each destination had its own service and queue.
- A shared-library change meant deploying 140+ services.
- Uneven load and growing operational overhead.
- Merged into one service: productivity went up.

Same code, same team: splitting added overhead, not autonomy.

Prime Video · 2023

Distributed monitoring tool → one process

- A stream-quality monitor built from serverless steps.
- Video frames moved between steps through storage.
- Hit scaling limits; per-step costs added up.
- Merged into one process: infrastructure cost down >90%.

Steps that exchange heavy data constantly belong together.

Microservices are wrong for the wrong problem, even at large companies.

It is mostly about organization

"Microservices buy you options."

— James Lewis

Technical pain?

Often cheaper fixes exist: scale out the monolith (X-axis), add caching, modularize the code.

Organizational pain?

Many teams blocking each other is the problem microservices solve best.

The price

More network calls, harder data consistency, more to operate and monitor.

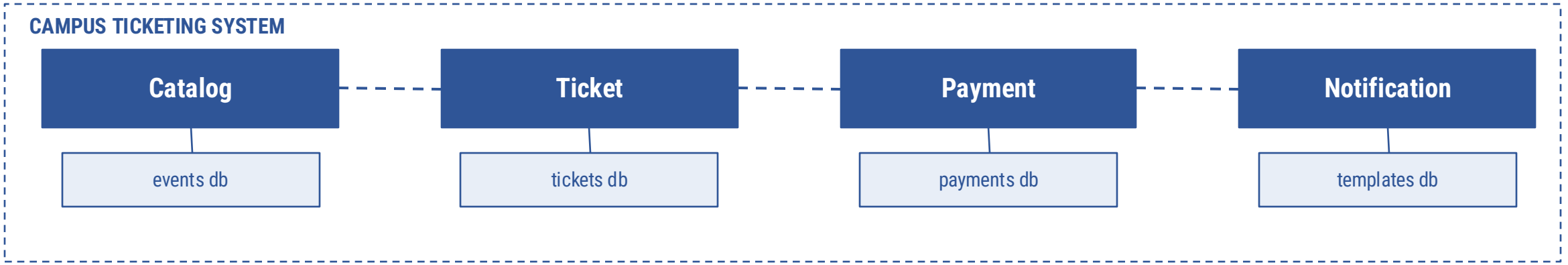
Pay for options only when you plan to use them.

What microservices are

The definition, and the properties that make a service a microservice.

What are microservices?

Independently releasable services, modeled around a business domain, that work together over a network.



Independently releasable

Change and deploy one service alone.

Business-shaped

Each service maps to a business capability.

Owns its data

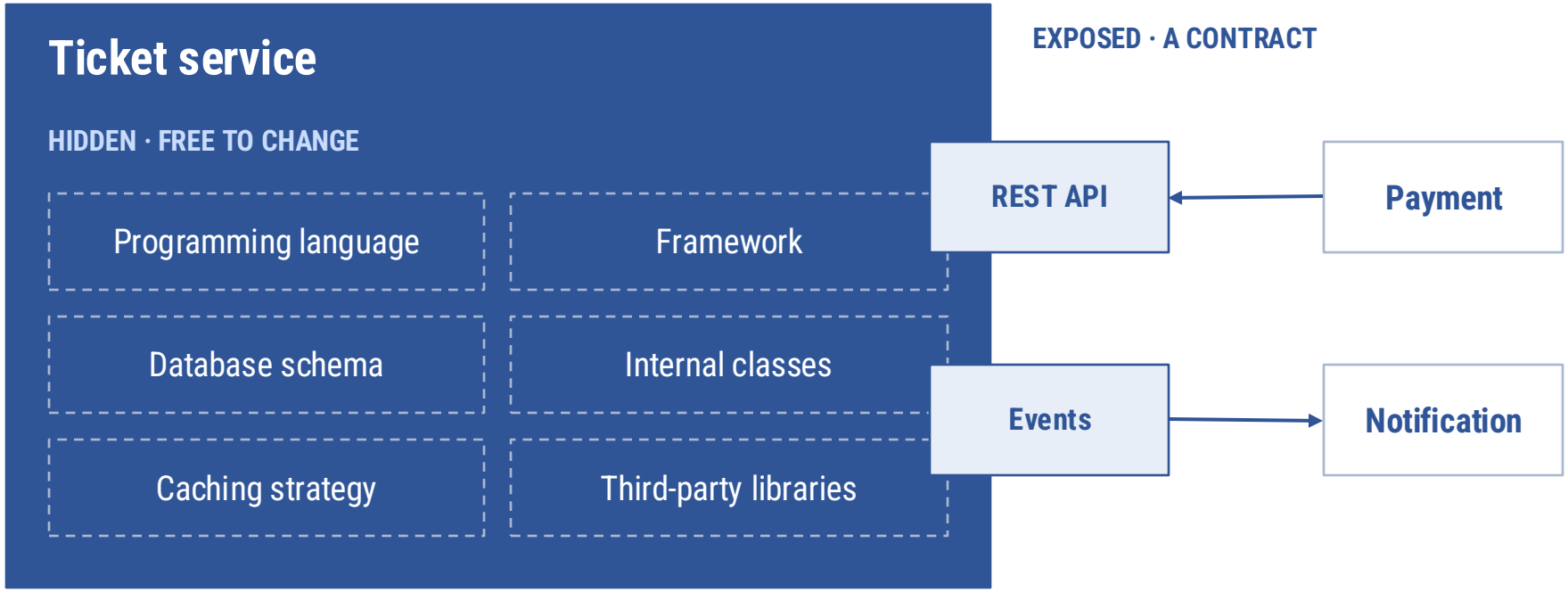
Others ask through its interface.

Together they are one system. Each one can change alone.

Information hiding

Hide as much as possible inside.
Expose as little as possible outside.

Consumers only see the interface. They should never know or care what is behind it.



What is hidden can change freely. What is exposed becomes a contract.

Independent deployability



To make this possible

Stable contracts

Consumers depend on the interface, not internals.

No shared database

Schema changes stay inside one service.

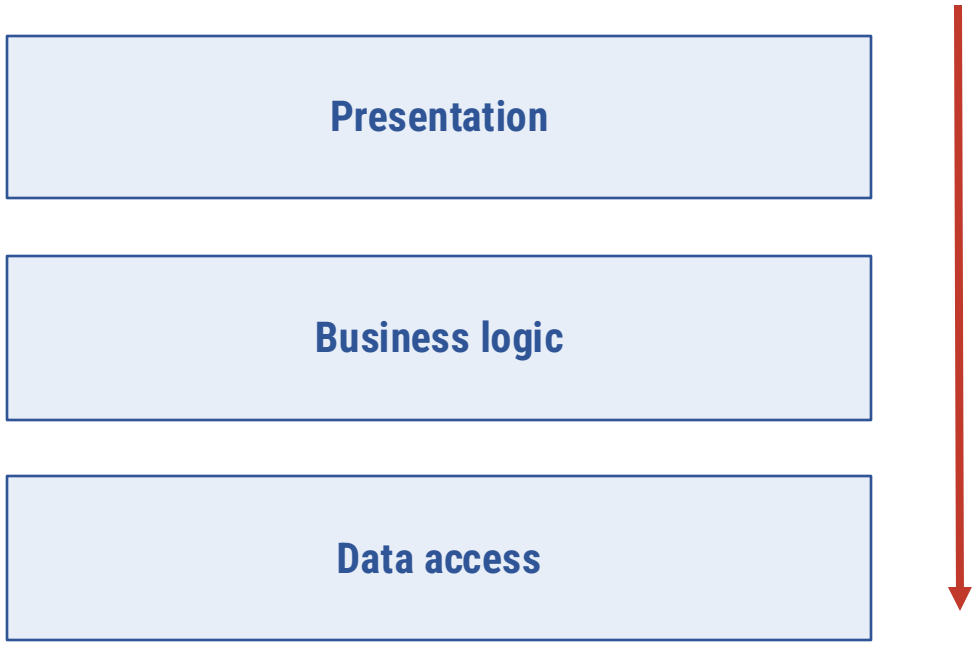
Backward-compatible changes

Old consumers keep working after a deploy.

If two services must be deployed together, they are not independent.

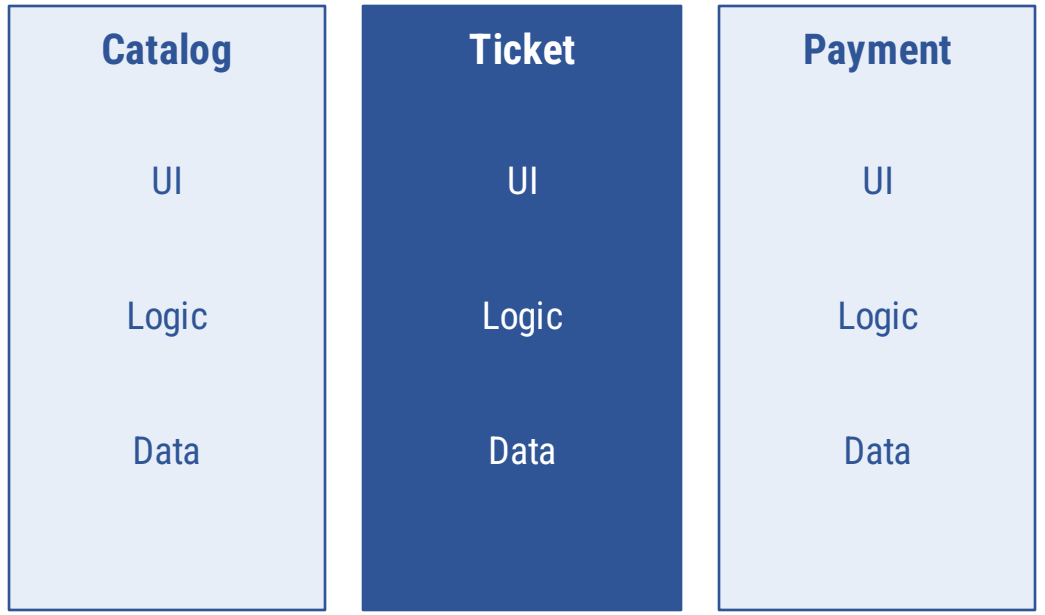
Modeled around a business domain

SPLIT BY TECHNICAL LAYER



"Add student discount" touches every layer

SPLIT BY BUSINESS CAPABILITY

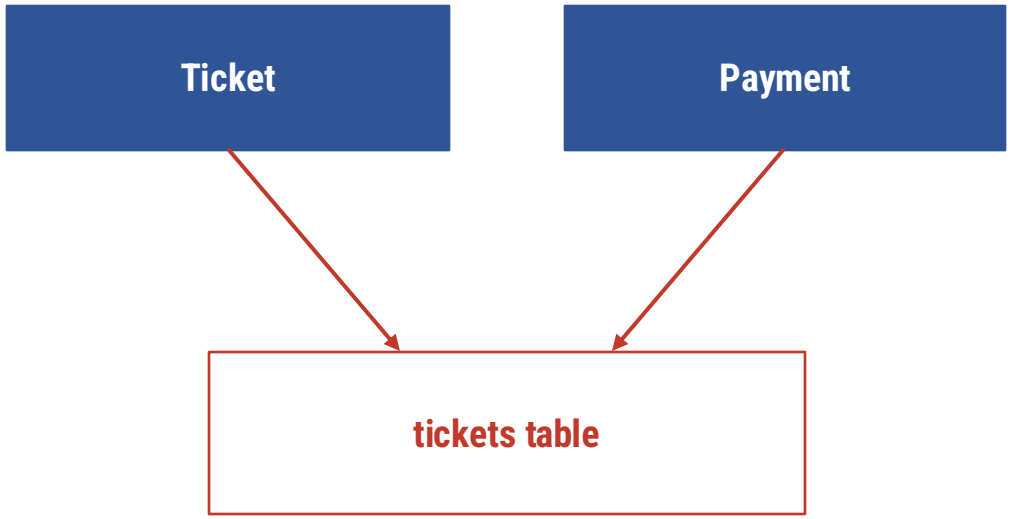


The same change stays inside Ticket

Group what changes together: cohesion of business function beats cohesion of technology.

Owning its own state

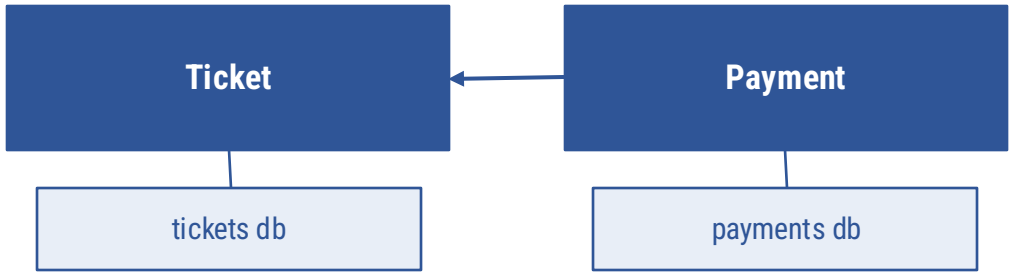
SHARED DATABASE



Ticket renames a column → Payment breaks.
Who owns the data?

EACH SERVICE OWNS ITS DATA

GET /tickets/42



Ticket changes its schema freely.
The API is the only contract.

Ask the service. Don't query its tables.

How big should a microservice be?

"A microservice should be as big as my head."

— James Lewis



Nobody understands it all. Teams collide again.

One team understands, owns and changes it. Most changes stay inside.

Chatty "nanoservices". Every feature crosses the network.

Not a useful measure: lines of code, number of endpoints, number of classes.

Size a service by understandability and ownership, not by lines of code.

Finding service boundaries

Where to cut decides almost everything that follows.

Coupling and cohesion

High cohesion

Code that changes together, stays together.

Loose coupling

A change in one service rarely forces a change in another.

TYPES OF COUPLING, FROM LOOSE TO TIGHT

Domain	Pass-through	Common	Content
Order calls Payment to charge a card. Normal; keep it small.	A passes data to B only because C needs it.	Several services read and write the same shared data.	A service writes another service's tables directly.

looser — tighter

Good boundaries keep coupling at the domain level and nothing tighter.

Domain-Driven Design in one slide

Ubiquitous language

Use the words domain experts use, in conversations and in code.

Ticketing: "reserve a seat", not "insert row".

Aggregate

A cluster of objects treated as one unit with its own lifecycle and consistency rules.

Ticketing: an Order with its line items.

Bounded context

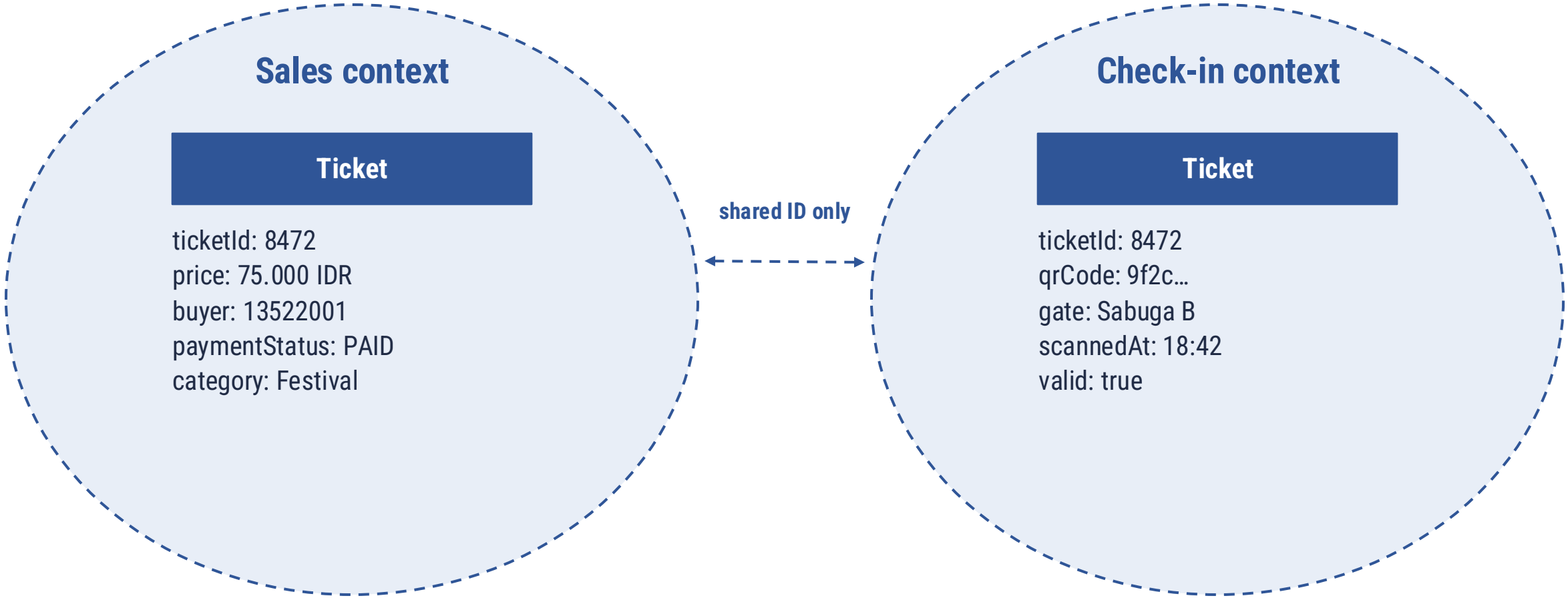
A boundary inside which a model and its terms are consistent; details are hidden outside.

Ticketing: Sales, Check-in, Payment.

Source: Eric Evans, Domain-Driven Design (2003)

Starting point: one bounded context ≈ one service, or a few.

Bounded context: one word, two models



Same word, different model. Don't force one giant Ticket class on everyone.

Decomposition strategies

1

By business capability

What the business does: sell tickets, take payments, notify buyers.

2

By subdomain

Core (ticket sales), supporting (check-in), generic (email). Generic ones can be bought.

3

By volatility

Split out the parts that change often, so stable parts stay stable.

4

By team

Each team owns a few services end to end. Conway's Law in action.

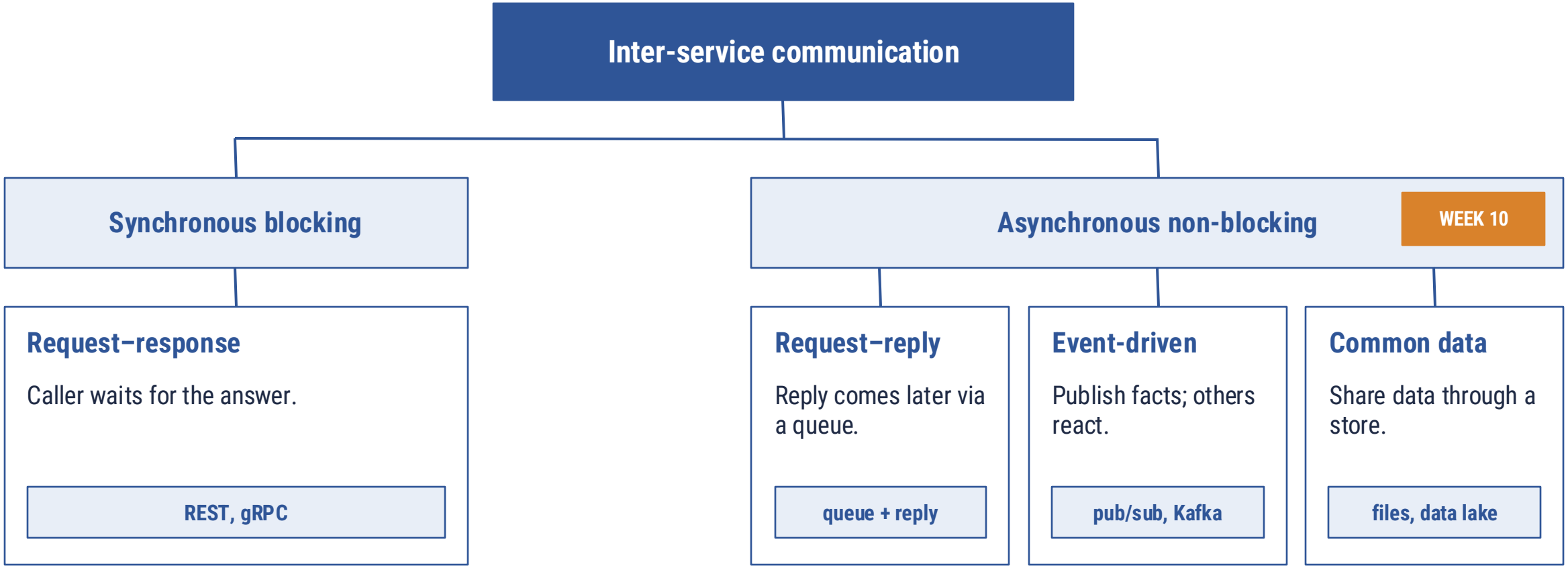
Rule of thumb: data that must be strongly consistent together usually belongs in one service.

Use several lenses together. No single rule gives the right boundary.

How services communicate

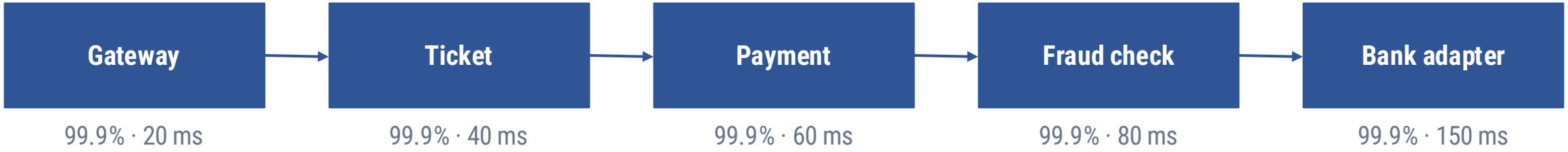
Choose the style first, then the technology.

Communication styles



Pick the communication style first, the technology second.

Synchronous communication between services



Availability

99.5%

$0.999^5 \approx 0.995 \rightarrow$ about 3.6 h downtime per month, instead of 44 min for one service

Client Latency

350 ms

Every hop adds network time and serialization. Slow outliers (p99) add up even worse.

Every synchronous hop multiplies failure and adds latency.

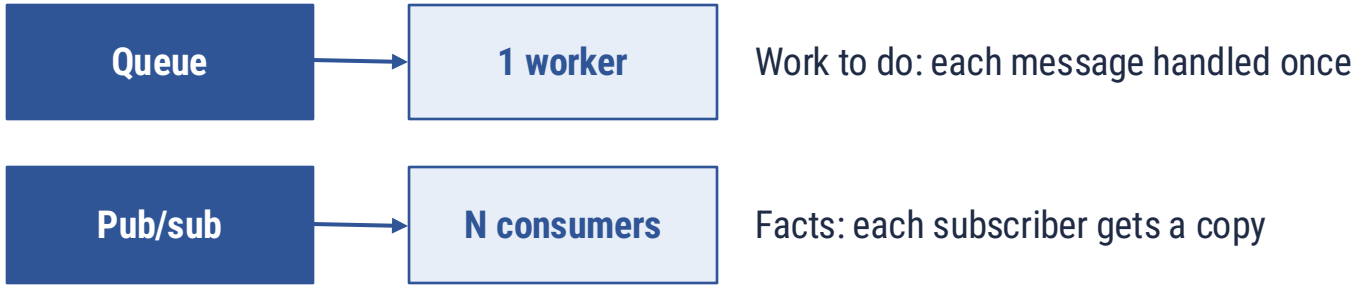
Synchronous options: REST, gRPC, GraphQL

	REST (HTTP + JSON)	gRPC	GraphQL
Contract	OpenAPI, optional	.proto schema, required	GraphQL schema, required
Payload	JSON text	Protocol Buffers, binary	JSON, shaped by the query
Strengths	Universal, cacheable, easy to debug	Fast, typed, streaming, code generation	Client picks fields; one round trip
Watch out	Over- or under-fetching; schema not enforced	Needs HTTP/2; awkward from browsers	Caching is harder; costly queries
Typical use	Public APIs and most service APIs	Internal service-to-service calls	Backend for a rich UI

Default to REST; reach for gRPC inside, GraphQL for flexible UIs.

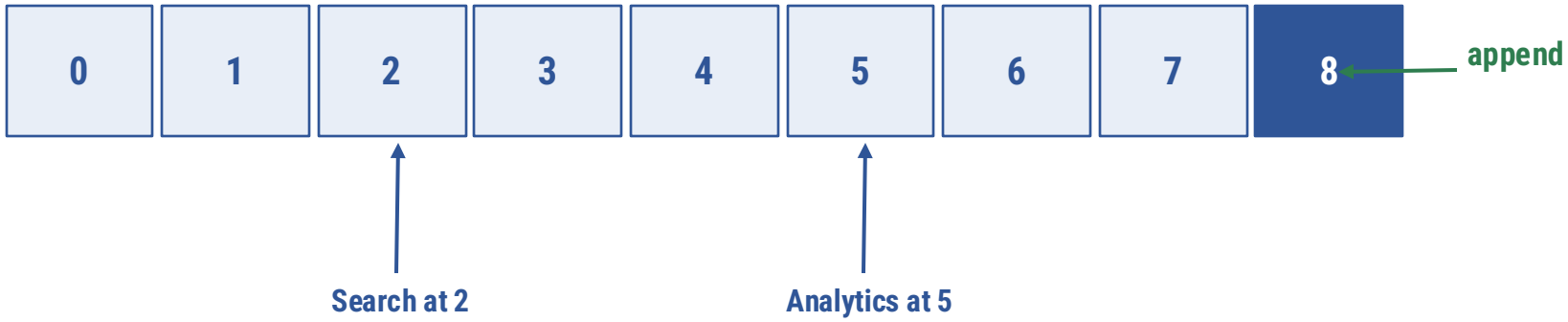
Asynchronous communication between services

RECAP · WEEK 10



NEW · THE LOG (e.g., APACHE KAFKA)

topic: ticket-events · partition 0



Why a log?

Events are kept, not deleted on delivery.

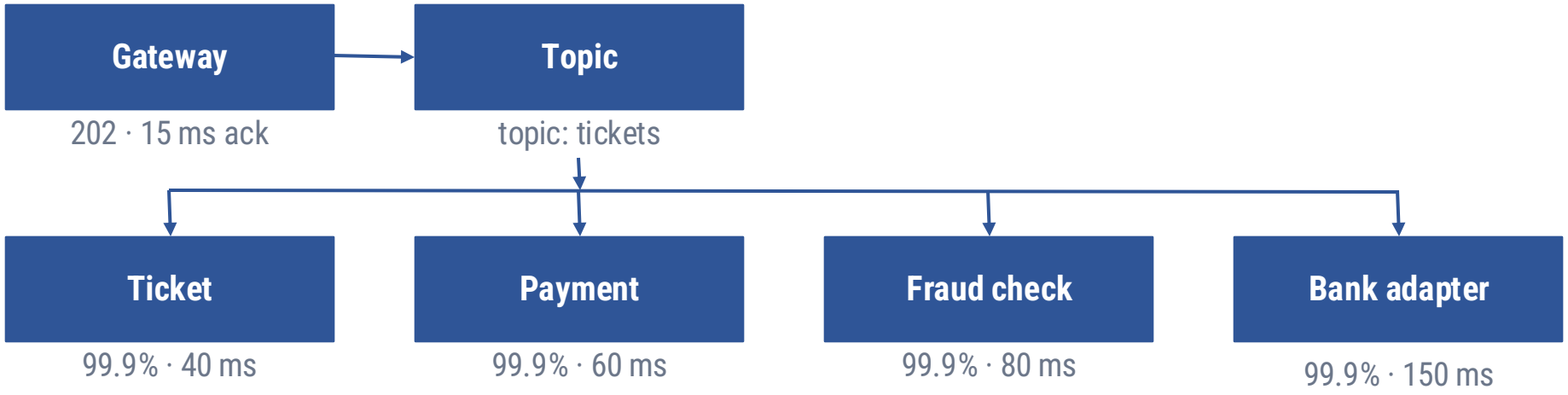
Each consumer reads at its own pace.

A new service can replay history.

Feeds real-time streams instead of nightly batches.

Queue: work to do. Log: a history of facts many services can replay.

The ticket flow, as asynchronous events



Availability

99.9%

The gateway's ack no longer waits on Ticket, Payment, Fraud check, or Bank adapter — a slow consumer doesn't drag it down.

Client Latency

15 ms

That's just the publish-and-ack. The four consumers finish their work in the background, each at its own pace.

An event decouples failure and latency: hops no longer multiply each other.

Contracts and versioning

Safe

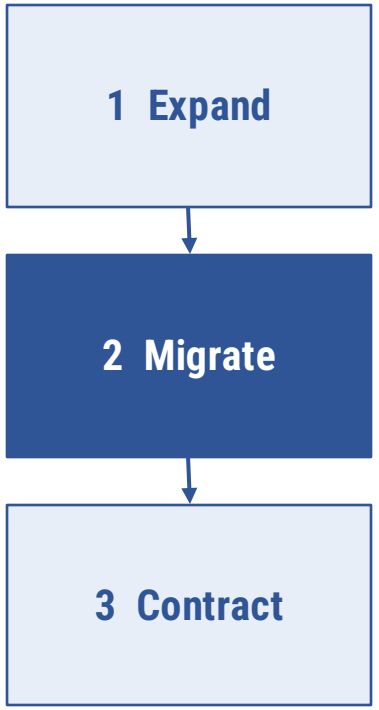
- Add an optional field
- Add a new endpoint
- Add a new event type

Breaking

- Remove or rename a field
- Change a type or meaning
- Make an optional field required

Tolerant reader
Consumers ignore unknown fields and read only what they need.

Expand → migrate → contract



Add phone alongside the old contact field. Both work.

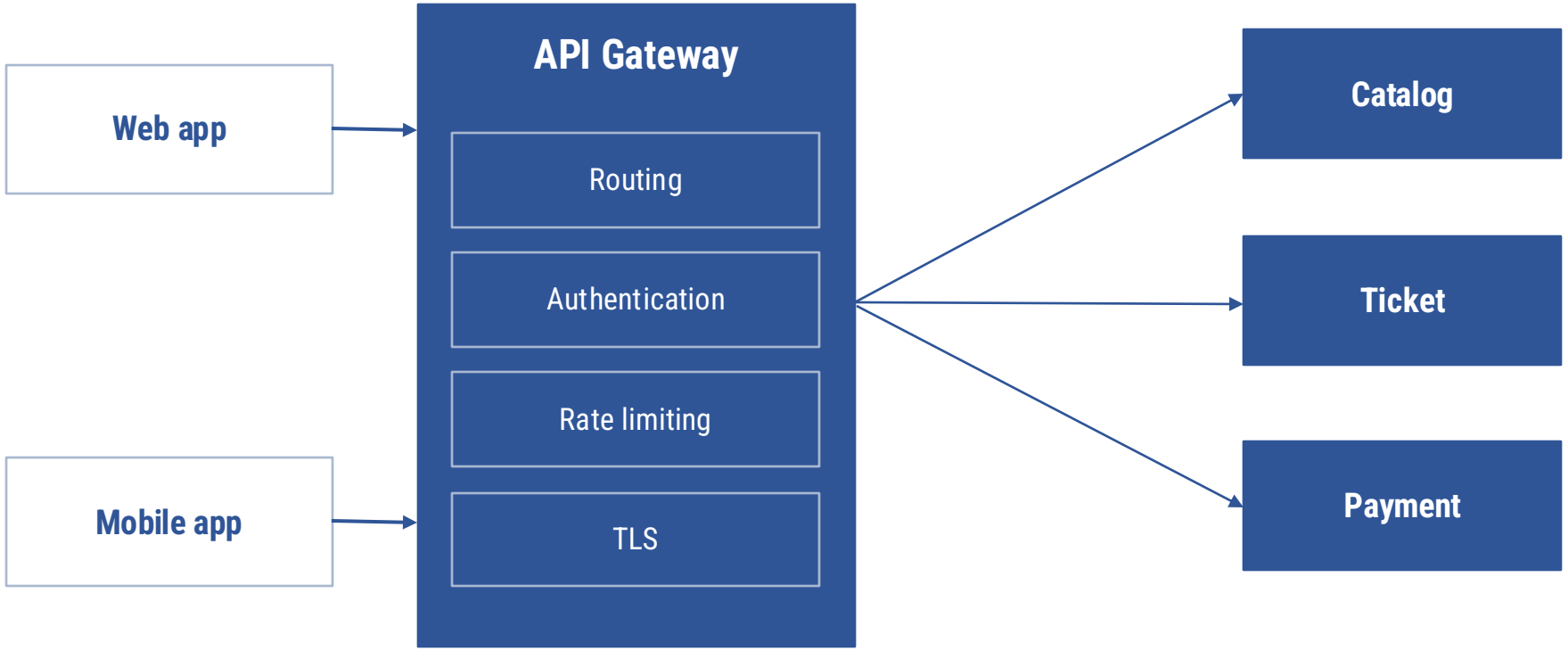
Consumers move to the new field at their own pace.

Once nobody reads it, remove the old field.

Versioned endpoints (/v2) are a last resort: then run both versions.

Change contracts so that old consumers keep working.

API Gateway

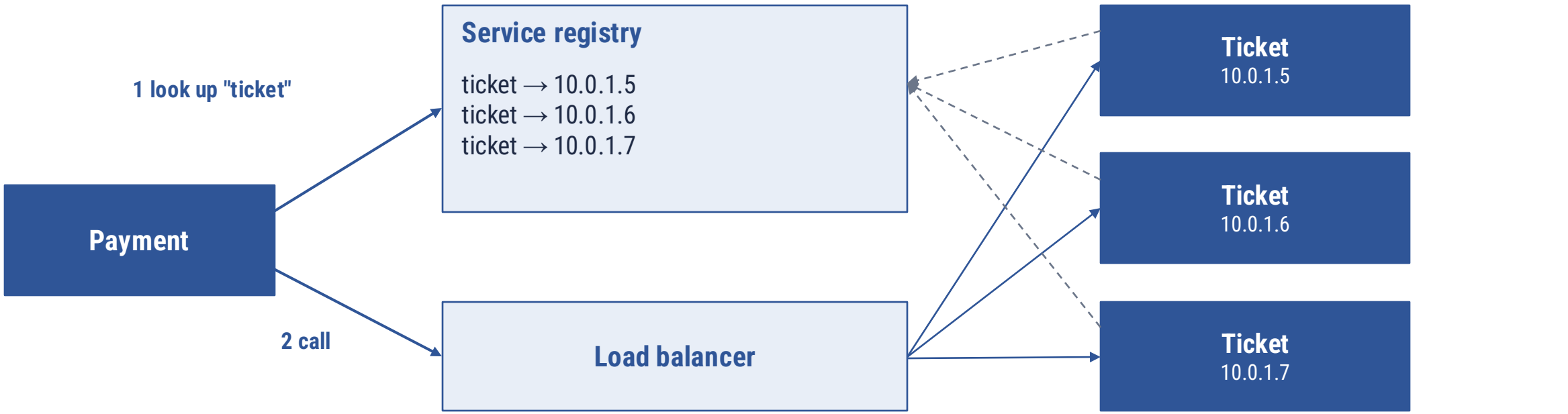


Gateway: edge concerns for all clients.

Avoid: business rules in the gateway. It becomes a new monolith that every team must change.

Gateway for cross-cutting edge concerns. Business logic stays in services.

Service discovery and load balancing



Client-side discovery

Caller asks the registry and picks an instance itself.

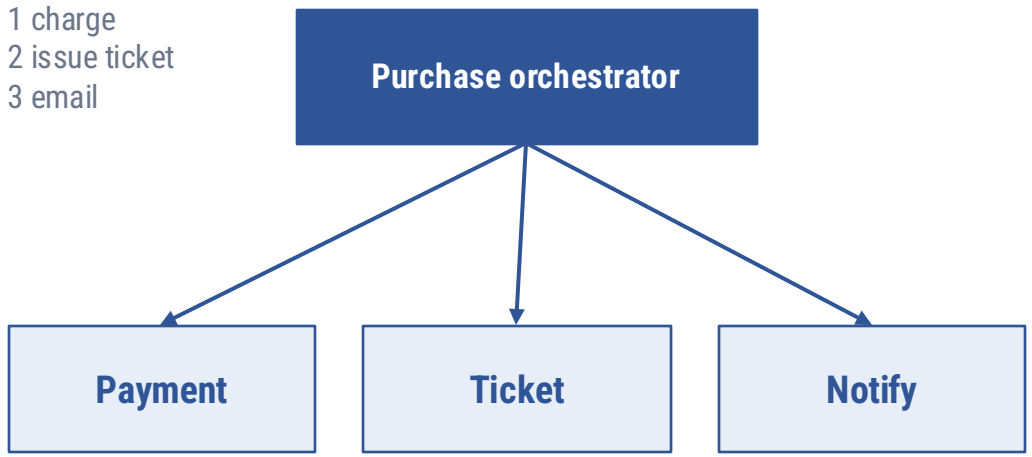
Server-side discovery

Caller uses a stable name; a load balancer or DNS routes it.

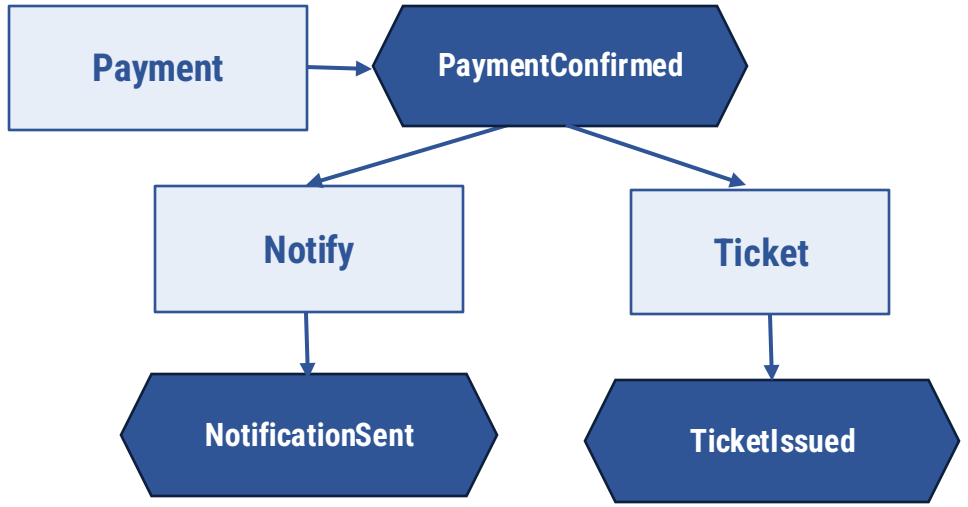
Instances come and go. Callers need a stable name, not an IP address.

Orchestration vs. choreographyx

ORCHESTRATION · "PLEASE DO"



CHOREOGRAPHY · "THIS HAPPENED"



Orchestration	Choreography
+ Workflow visible in one place; easy to follow and change	+ Loosely coupled; new consumers subscribe without changes
− Orchestrator can grow into a central "god service"	− Workflow is implicit; harder to see and trace

Neither is forbidden. Choose per workflow, and keep the trade-off visible.

06

Should you use microservices?

Now that we know the mechanics, weigh the costs honestly.

The costs

Developer experience

Can't run 30 services on one laptop.

team practice

Technology overload

Every team is tempted by a new tool.

team practice

Infrastructure cost

More processes, network, storage, licences.

later: cloud

Latency

Every hop adds network time.

§5 communication

Data consistency

No transaction across services.

§6 sagas, outbox

Reporting

Data is spread across many databases.

§6 read models

Testing & security

More moving parts, more endpoints to protect.

later lecture

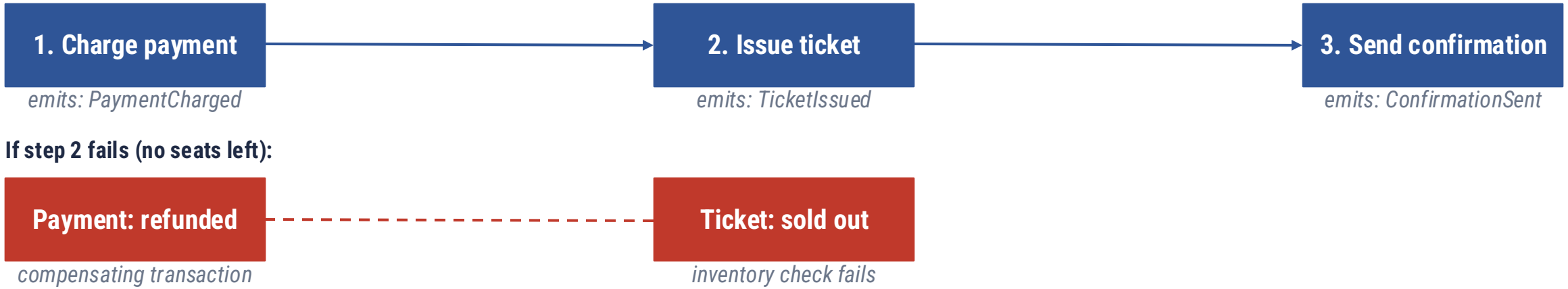
Monitoring

Which of 30 services is slow?

later: observability

Every benefit from section 2 comes with a bill. Read it before you sign.

The ticket purchase, as a saga (Pattern)



No distributed lock

3 local txns

Each step commits its own local transaction and publishes an event; no lock spans Payment, Ticket, and Notify.

On failure

Compensate

Finished steps run a compensating transaction instead of leaving the system half-done — here, refunding the charge.

A saga trades one big transaction for many small ones, each with its own compensating action.

Monolith first?

Stay with a monolith when

- The team is small
- The product is new and still changing
- Domain boundaries are unclear
- No deployment automation yet

Start modular

Structure the monolith into modules along business domains, with clear interfaces and no reaching into each other's tables.

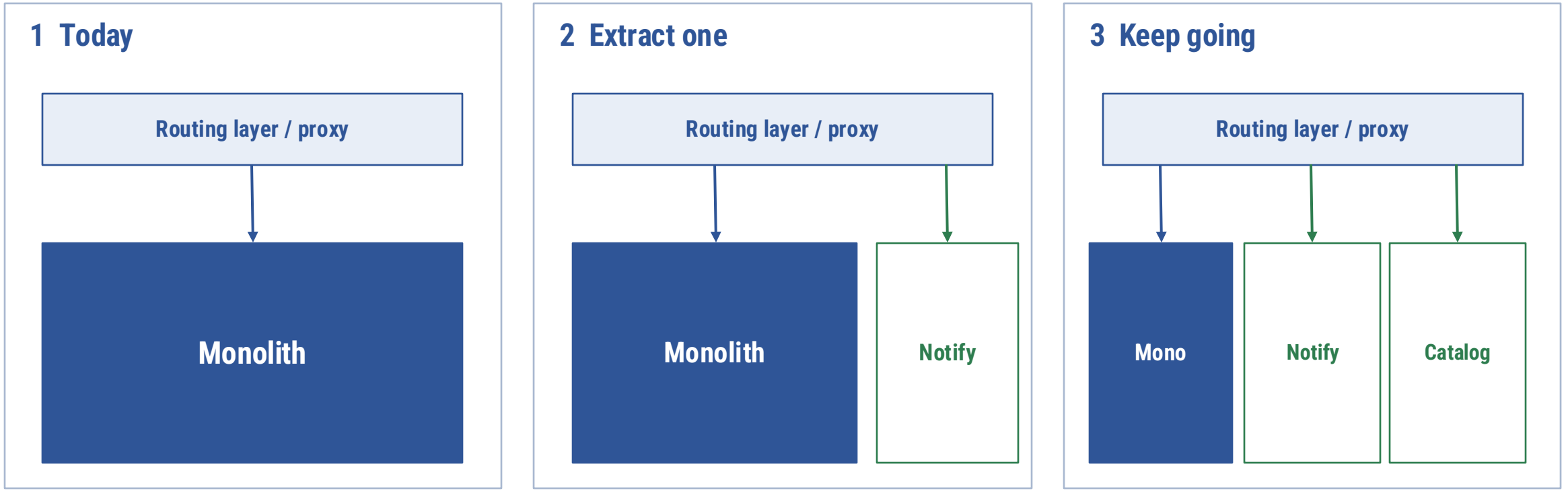
You get most of the team benefits now, and an easy path to extract a service later.

Consider microservices when

- Many teams block each other
- Parts need very different scaling or availability
- Boundaries are clear and stable
- You can automate and operate distributed systems

Microservices are an architectural approach, not the architectural approach.

Migrating: the strangler fig



Branch by abstraction
Same idea inside the code: interface → new implementation → switch → delete old.

Start small
Pick a well-bounded, low-risk capability first. Split data last, carefully.

Migrate incrementally. Never a big-bang rewrite.

Anti-patterns to avoid

01

Distributed monolith

Design for independent deploys.

02

Too many tiny services

Merge until each one is meaningful.

03

Chatty services

Coarser APIs; rethink the boundary.

04

Shared database

Exactly one owner for each piece of data.

05

Mixed responsibilities

One business capability per service.

06

Spaghetti dependencies

Clear boundaries; no call cycles.

07

Ignoring observability

Logs, metrics, traces: next lecture.

08

Coupling to internals

Depend on contracts, not internals.

09

Teams not aligned

Align ownership with services (Conway).

10

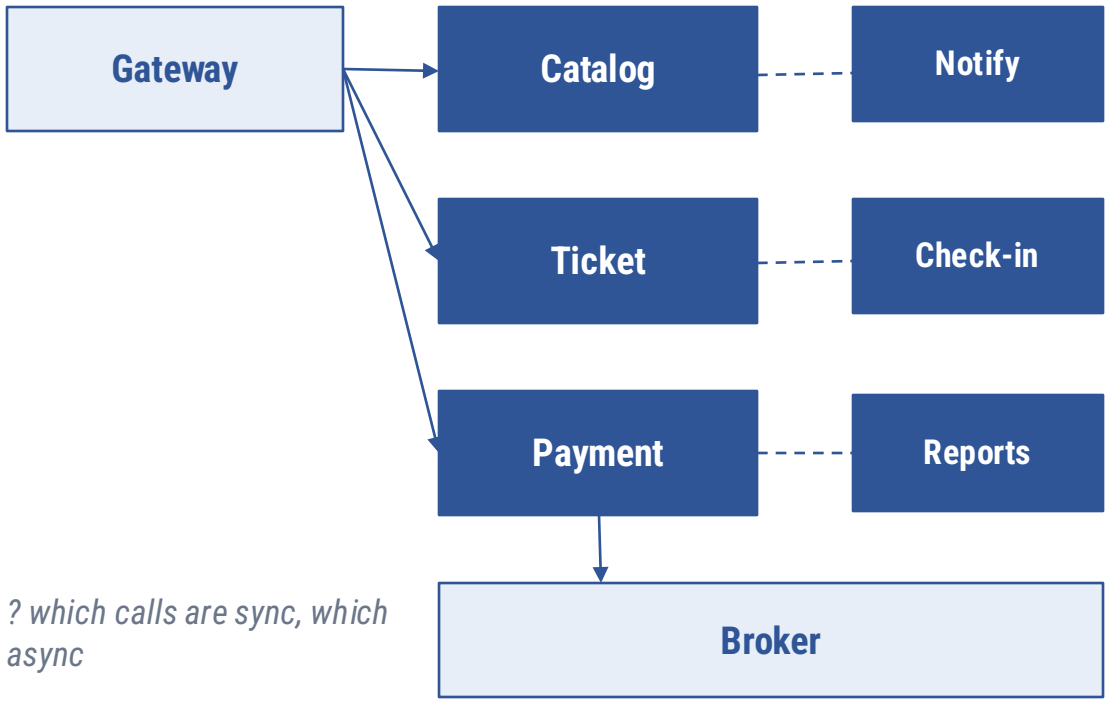
Premature adoption

Monolith first, modular from day one.

Most anti-patterns are a boundary in the wrong place, or no boundary at all.

Exit ticket: campus ticketing

YOUR DESIGN FROM THE PAIR ACTIVITY



? which calls are sync, which async

ON YOUR OWN · 3 MIN

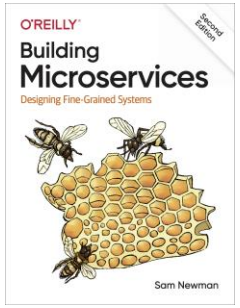
1. One synchronous call you would make asynchronous, and why
2. One saga step and its compensating action
3. One piece of data and its single owning service

SPLIT BY BUSINESS | **OWN YOUR DATA** | **DESIGN FOR FAILURE**

Summary and references

Key takeaways

1. Microservices solve mostly an organizational problem: many teams, one system.
2. Split by business capability; each service owns its data.
3. Prefer async between services; every sync hop costs availability.
4. Use sagas and the outbox instead of distributed transactions.
5. Start with a modular monolith; migrate with the strangler fig.



References

- Sam Newman, Building Microservices, 2nd ed. (O'Reilly, 2021): ch. 1–6, 12, 15–16
- Eric Evans, Domain-Driven Design (2003)
- Abbott & Fisher, The Art of Scalability: the scale cube
- Chris Richardson, microservices.io: saga, outbox, API gateway patterns
- Martin Fowler, martinfowler.com: Microservices; StranglerFigApplication
- Werner Vogels, "A Conversation with Werner Vogels", ACM Queue (2006)
- Segment Engineering, "Goodbye Microservices" (2018); Prime Video Tech blog (2023)

Next: containers & deployment · testing & security · observability · cloud & serverless

Split by business, own your data, design for failure, and only when the pain is real.